Laboratory 1: Short project

Docker, Compose and Swarm — high availability edition

Marc De Rialp David Franco Omar Antonio Cornejo David Madrid
Victor Yakovlev

Cloud Computing and Distributed Systems
MSc in Machine Learning and Cybersecurity for Internet Connected Systems, UPC

October 2026

Abstract

We containerised the digital infrastructure of a company with Docker: a shared stack (web front, certificates, shared disk), a reviews application with its database, cache, mail relay and backups, a monitoring platform, and the Moodle learning management system. The same project runs in two environments: a local one on a single machine (`compose.yml`) and a production one on a Docker Swarm of seven nodes (`production.yml`), in which every user-facing service is replicated, the database and the cache are clustered and fail over by themselves, and the backups are encrypted and kept on a node of their own. The images are published in Docker Hub, so deploying only needs to pull and start them. The report describes the design, the deployment, the management of data and backups, the monitoring, the test cases and their results.

Contents

1 Introduction **3**

2 Architecture **3**
2.1 Nodes . . . 3
2.2 Services . . . 3

3 Images and design decisions **4**

4 Scalability and high availability **6**
4.1 Replicas . . . 6
4.2 Failures . . . 6
4.3 Updates . . . 6

5 Deployment **6**

6 Access, users and passwords **7**
6.1 Web addresses . . . 7
6.2 Users and passwords . . . 8

7 Data management and backups **8**

8 Observability **9**

9 Security **9**

10 Test cases **10**
10.1 Local environment . . . 10
10.2 Production environment . . . 10

11 Results **11**
11.1 Local environment . . . 11
11.2 Production environment (`production.yml`) . . . 11

12 Limitations **12**

13 Conclusions **12**

1 Introduction

We containerised the infrastructure of Figure 1 of the statement: the shared services (*basic*), a company application (*app*), an observability platform (*stats*) and Moodle (*atenea*). As the core application we wrote a review site in Flask: people leave a comment with a rating from 1 to 5, see the distribution and the latest reviews, each new comment sends a mail through our SMTP relay and an automatic feeder collects ratings published in other web pages.

This version goes beyond a single working deployment: every user-facing piece has at least two replicas on different nodes, the application database and the cache are clustered, the backups are encrypted and kept on a node of their own, and every service is monitored. Everything is deployed with one `docker compose up` (local) or one `docker stack deploy` (production), with the images already published in Docker Hub.

The repository contains `compose.yml` (local), `production.yml` (swarm), one folder per image with its Dockerfile and files, `scripts/` (node labels, publication, simulated swarm and restore) and this document. The local environment has no TLS and uses named volumes, but it contains the same clusters (database, cache) as production, so they can be tested on one machine.

2 Architecture

2.1 Nodes

The swarm has seven nodes; the placement is done with the node label `role` (and `slot` for the two app nodes, to keep a primary and its replica apart).

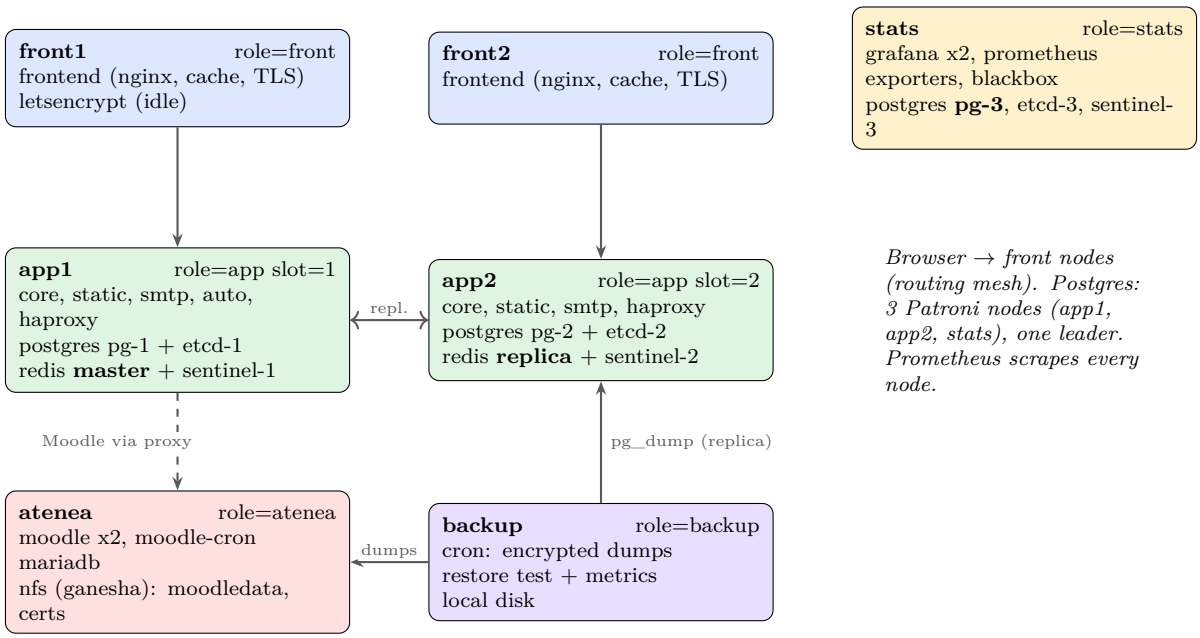


Figure 1: Nodes and services (simplified). The routing mesh accepts the traffic on both front nodes; `backup` pulls from a replica, from MariaDB and from the NFS volume.

2.2 Services

Table 1 maps the boxes of the diagram of the statement to our services, the node where they run and their replicas. Each stack is an encrypted overlay network and a service only joins the networks it needs: the databases are not in the `proxy` network, so the frontend can not reach them.

Public names in production: `app.<domain>`, `lms.<domain>` and `grafana.<domain>`; with the default `DOMAIN=localhost` they are `app.localhost`, `lms.localhost` and `grafana.localhost` on

Stack	Service	Nodes	Repl.	Description
basic	`frontend`	front	2	nginx: only public service (80/443), cache, TLS, proxy by subdomain
	`letsencrypt`	front	1	certbot; idle with a local domain (self-signed certificate)
	`nfs`	atenea	1	NFSv4 server (NFS-Ganesha) inside the stack, port 2049 in host mode
app	`core`	app	2	Flask + gunicorn, reviews
	`static`	app	2	Apache httpd, `/static/`
	`pg-1..3`	app1, app2, stats	3	PostgreSQL 16 + Patroni (leader + 2 replicas)
	`etcd-1..3`	app1, app2, stats	3	consensus store of Patroni
	`db`	app	2	HAProxy: `db:5432` leader, `db:5433` replicas
	`redis-1`, `redis-2`	app1, app2	1+1	cache: master and replica
	`sentinel-1..3`	app1, app2, stats	3	failover of the cache (quorum 2)
	`smtp`	app	2	Postfix relaying the mails (to Mailpit by default)
	`mailpit`	app	1	mail catcher with a web page (`mail.localhost:8080`)
	`auto`	app	1	feeder of external ratings
	`backup`	backup	1	encrypted dumps + restore test
atenea	`moodle`	atenea	2	Moodle 4.5.14 LTS
	`moodle-cron`	atenea	1	same image, `cron.php` every minute
	`moodle-db`	atenea	1	MariaDB 11.4
stats	`grafana`	stats	2	dashboards as code + Infinity plugin (the "?")
	`prometheus`	stats, backup	2	metrics and alert rules, one TSDB per replica
	exporters	stats / global	—	postgres, redis, mysqld, nginx, blackbox, node, cAdvisor

Table 1: Services of the infrastructure.

port 443. Locally the same names are used with `localhost` on port 8080 (browsers resolve `*.localhost` to 127.0.0.1).

3 Images and design decisions

Frontend and cache. `nginx:alpine` with the template feature of the official image (two templates, local and production). Pages of the app are cached for 10 s in nginx (`proxy_cache`, `proxy_cache_lock` and `proxy_cache_use_stale`, so users still get an answer while the app is restarting); static files for one hour. A user that has just posted a review receives the cookie `posted` (10 s) which bypasses the cache, so everybody sees their own review at once. The footer at the bottom left of the page shows *front: <node> / app: <node>*. The name of the node comes from the swarm template `{{.Node.Hostname}}`; the front node is requested by JavaScript to `/__node` (not cached) because the HTML itself may be served from the cache. When the page is refreshed the routing mesh may hand the request to the other frontend replica and the footer changes.

TLS on 443. Production serves HTTPS on 443 with a **self-signed certificate** on `localhost`: nginx cannot start without a certificate, so the entrypoint generates one (key protected with the group passphrase `groupH`, `CERT_PASSWORD`, loaded with `ssl_password_file`) and the browser shows a warning that has to be accepted. The `letsencrypt` service of the diagram is part of the stack and has the full logic (`certbot certonly -webroot`, retries, renewal and the switch of nginx without a restart), but with a local name it stays idle, because a certificate authority can only validate a public domain.

Core application. Python 3.12 slim, non-root, two replicas. The page shows the average, the distribution of ratings, a paginated list and a form with star rating; the average is also published as JSON-LD (schema.org), which the feeder reads. The data of each page is cached in Redis under a key that contains a version number: a new review increments the version, so stale entries are never read again. If Redis is down the page still works, because every cache call is inside `try/except`; if the database is not ready the answer is a clean 503. `/healthz` (liveness) and `/readyz` (database) are used by the healthchecks and by the probes. Writes go to `db:5432` (the PostgreSQL leader) and reads to `db:5433` (the replicas).

PostgreSQL cluster (Patroni). Three PostgreSQL 16 nodes (`pg-1`, `pg-2`, `pg-3`) on app1, app2 and stats, each managed by **Patroni** with a three-member **etcd** (`etcd-1..3`, on the same nodes) as consensus store. Patroni initialises the first node, builds the others as streaming replicas (`pg_basebackup`) and holds the leader key in etcd; if the leader disappears for `ttl` = 30 s a replica takes the key and is promoted, and the old leader is resynchronised with `pg_rewind` when it comes back. Three nodes give a majority for etcd and for the election even if one node is lost, which two nodes could not. `db` is HAProxy (two replicas): it asks the REST API of Patroni which node is the leader (`/primary`) and which are healthy replicas (`/replica`) and exposes one stable address, `db:5432` for the leader and `db:5433` for the replicas. Core, Grafana and the backups use only that name, so a failover needs no change anywhere. Tables and roles are created by the `post_bootstrap` script of Patroni on the first node, not by the application, so several replicas of core starting together cannot collide. Patroni uses a *fast* checkpoint for the base backup of a new replica, otherwise it waits for the spread checkpoint (minutes).

Redis cluster. `redis-1` (master) and `redis-2` (replica) with three Sentinels (app1, app2, stats; quorum 2). The application asks the Sentinels for the master (`redis.sentinel.Sentinel`), so after a failover it finds the new one by itself. The cache is optional: if Redis disappears the application keeps working, only slower.

Moodle. `php:8.3-apache`, release `4.5.14` *pinned* with its SHA-256: the file is downloaded with retries, verified and only then extracted, in its own layer. The entrypoint waits for MariaDB and installs Moodle from the CLI the first time. Only one container installs: a lock directory in the shared moodledata is the mutex and the installer touches a heartbeat file every 10 s; if the heartbeat is older than 90 s the installer died and another container takes the lock (atomic `mv`), empties the half-built schema and starts again. The installation is only trusted once the file `.installed` exists. Sessions are kept in the database, so any replica serves any user; `moodle-cron` is the same image running `cron.php` in a loop.

NFS inside the stack. A swarm service cannot be `privileged`, so instead of the kernel server we use **NFS-Ganesha** in user space, which only needs two capabilities (`DAC_READ_SEARCH`, `SYS_RESOURCE`, `cap_add` of the stack file) and no kernel module. It runs on the atenea node with a local volume and exports `moodledata`, `letsencrypt` and `acme` (NFSv4 only, port 2049). The port is published in `host` mode on that node only and the other nodes mount it at its private address (`NFS_SERVER`); a task that starts before the server is retried by swarm.

Backups and observability. See Sections 7 and 8.

Scope of "production". Production means the complete architecture, not a public service: it runs on our own computer (the seven nodes are simulated with Docker-in-Docker, Section 11.2), it is not opened to the Internet and it has no public domain, so the certificate is self-signed and 443 is served on `localhost`. A deployment on a public server with a real domain would only need `DOMAIN` and `LE_EMAIL`, and certbot would take over; that case is outside this project.

4 Scalability and high availability

4.1 Replicas

The statement asks every service to be scalable, except cron jobs and DBMS. The stateless services run with two replicas (one per node) and can be scaled with `docker service scale`; Prometheus discovers new replicas with the DNS name `tasks.<service>`. The services with one replica are: `letsencrypt`, `backup`, `moodle-cron` and `auto` (periodic jobs: a second copy would only repeat the work), `moodle-db` (DBMS), `nfs` (the shared disk) and `mailpit` (a development tool). PostgreSQL and Redis are DBMS, and are clustered instead of scaled (leader and replicas).

4.2 Failures

Failure	What happens	Recovery
a front node	the routing mesh sends all the traffic to the other `frontend`	automatic
a core / static replica	swarm reschedules; nginx serves `stale` cached pages meanwhile	automatic
Redis master	Sentinels promote the replica (≈4 s on an idle machine; about 30 s when the host was overloaded, because Sentinel enters its `TILT` protection mode); the app keeps working, the cache is optional	automatic
Postgres leader	Patroni elects a new leader among the two replicas and HAProxy follows it; writes fail only during the election (≈3 s if the container is stopped, ≈28 s if it is killed, the `ttl` of the leader key)	automatic
app node (slot 1)	one core left; pg-2 or pg-3 become leader, Redis replica is promoted, etcd keeps its majority (2 of 3)	automatic
stats node	monitoring is down; pg-3 and etcd-3 leave but the quorum holds (2 of 3)	node back
atenea node	Moodle, its database and the NFS (certs) are down; the app keeps working with the certificates it has loaded	node back, or restore from backup
backup node	no new backups; nothing else	node back

Table 2: Failure scenarios.

4.3 Updates

Updates go one replica at a time (`stop-first`, `parallelism: 1`) with `failure_action: rollback`, and are meaningful because the images we build have a `HEALTHCHECK`: a new replica must be healthy before the next one is replaced. The stateless services keep *exactly one replica per node* (`max_replicas_per_node: 1`), so a node failure never takes both; with that restriction there is no free slot to start the new replica before stopping the old one, and the other replica serves meanwhile.

5 Deployment

Local environment (one machine, no TLS):

```
docker compose pull && docker compose up -d
```

Production (seven nodes; `scripts/sim-swarm.sh up` does all this on one computer):

```
# once: build and publish the images
docker login && scripts/publish.sh

# on a manager, after the 7 nodes joined the swarm
scripts/label-nodes.sh # labels the nodes, prints the NFS_SERVER line
export NFS_SERVER=<private IP of atenea>
docker stack deploy -c production.yml lab1
```

All the images are in one public Docker Hub repository, `omarcornejo/lab1grouph`, with one tag per service (`<service>-<version>`), so no registry has to be started before deploying.

6 Access, users and passwords

For security and centralized configuration, credentials are injected with Docker secrets mounted at `/run/secrets`. For testing purposes within this laboratory setup, default passwords are standardized to `groupH`, the only exception is Moodle administrator, because Moodle refuses passwords that do not follow its policy (upper and lower case, digit, and symbol).

6.1 Web addresses

Service	Local (`compose.yml`)	Production (`production.yml`)	Auth
Reviews application	`http://app.localhost:8080`	`https://app.localhost`	Public
Moodle (Atenea)	`http://lms.localhost:8080`	`https://lms.localhost`	Required
Grafana	`http://grafana.localhost:8080`	`https://grafana.localhost`	Required
Mailpit	`http://mail.localhost:8080`	`http://mail.localhost:8080`	Public

Table 3: Deployed web endpoints and authentication requirements.

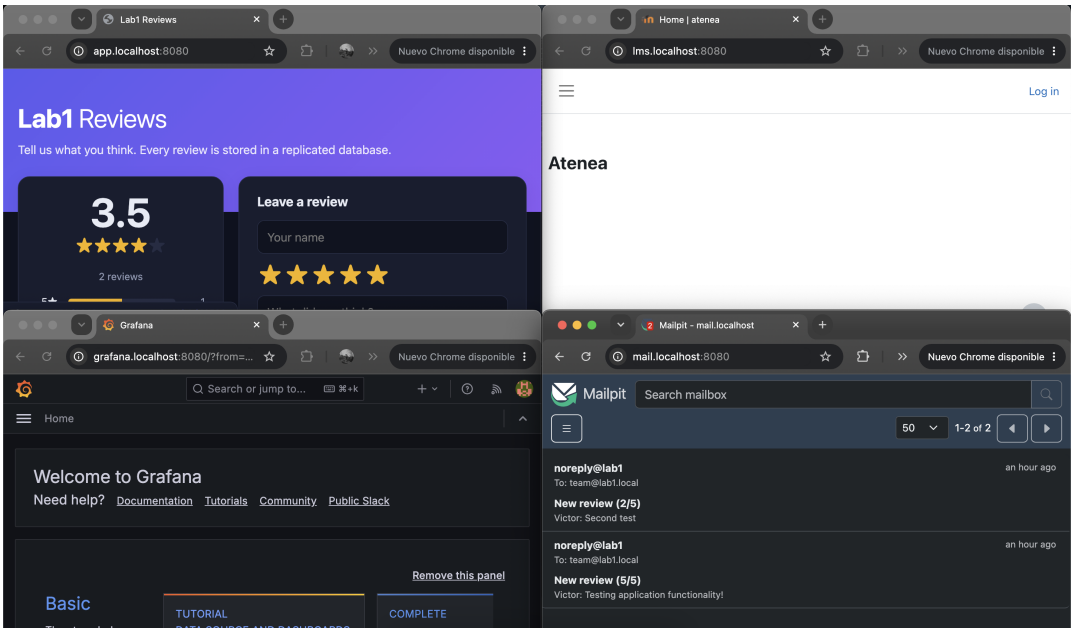


Figure 3: Running deployment of the web addresses.

6.2 Users and passwords

Table 4 shows the service credentials secret files, and the configured database access scopes.

Service	User	Password	Secret File
Grafana	admin	groupH	grafana_admin_password
Moodle	admin	groupH-2026	moodle_admin_password
PostgreSQL (Patroni)	postgres	groupH	pg_admin_password (Internal HA management)
PostgreSQL (App)	app	groupH	app_db_password (Owner of app DB)
PostgreSQL (Grafana)	grafana	groupH	grafana_db_password (Owner of grafana DB)
PostgreSQL (Replication)	replicator	groupH	replicator_password
PostgreSQL (Backup)	backup	groupH	backup_db_password (pg_read_all_data)
PostgreSQL (Metrics)	exporter	groupH	exporter_db_password (pg_monitor)
MariaDB (Moodle)	moodle	groupH	moodle_db_password
MariaDB (Auxiliary)	backup, exporter	groupH	Shared secrets (Read-only access)
Backup Encryption	—	groupH	backup_passphrase (AES-256-CBC)
Self-signed TLS key	—	groupH	CERT_PASSWORD
Feeder API	—	groupH	feed_token (Bearer Token)

Table 4: System account roles and Docker secret mappings.

7 Data management and backups

To avoid database corruption caused by network disruptions or file-locking conflicts, transactional DBMS files are stored directly on local node drives rather than over NFS.

Data Target	Volume Name	Physical Node Location
PostgreSQL Cluster & etcd	pg-1..3, etcd-1..3	Local disk (app1, app2, stats)
MariaDB & NFS Root	moodle-db, nfs-data	Local disk (atenea)
Moodle Data & Certificates	moodledata, certs, acme	NFS Storage (atenea)
Encrypted Backups	backups	Dedicated disk (backup)
Prometheus Replicas	Anonymous volumes	Local disk (stats, backup)
Redis Cache	In-Memory	Replicated RAM

Table 5: Physical storage distribution across Swarm nodes.

The backup service operates on a dedicated host isolated from core application nodes. Scheduled daily at 03:30, the pipeline performs:

1. **PostgreSQL Dump**: Uses pg_dump -Fc routed to read-only replicas (db:5433) under the backup role to prevent load on the HA leader.
2. **MariaDB Dump**: Executes mariadb-dump -single-transaction for transactional consistency.
3. **Moodle Data Storage**: Archives moodledata while stripping volatile session and cache directories.
4. **Encryption & Integrity**: Encrypts files using OpenSSL AES-256-CBC (PBKDF2) with the secret passphrase and generates matching SHA256SUMS.

Backups are initially written with a .partial suffix and atomic-renamed upon completion, and retention is configured for 14 days.

An automated integrity check runs every Sunday (and on demand with docker exec <backup> /restore-test.sh) The routine verifies checksums, decrypts archives, restores dumps into temporary containerized DBMS instances, and validates recovered row and table counts. Test outcomes are exported to Prometheus (backup_restore_test_ok).

8 Observability

Prometheus uses DNS service discovery (`tasks.`) so scaled replicas are scraped without manual changes. Dashboards are provisioned declaratively via Git, where Grafana visualizes node metrics alongside custom ratings using the Infinity plugin. Additionally, Prometheus alerting rules (`rules.yml`) flag offline endpoints, replication delays, missing cluster leaders, and backup failures. Table 6 summarizes the metric sources and monitored targets.

Metric Source	Monitored Target	Key Operational Indicators
`core`	Python Microservice	Requests/sec, posted reviews, Redis cache hit/miss ratio
Nginx Exporter	Frontend Proxy	Active connections, throughput per replica
Postgres Exporter / Patroni	PostgreSQL Nodes	Connection pool, transactions, replication lag, leader status
Redis Exporter	Cache Instances	Master/Replica role, memory consumption, hit ratio
MySQLd Exporter	MariaDB Instance	Active threads, queries per second (QPS)
Blackbox Exporter	Web Endpoints	Availability and HTTP response latency for Moodle/Grafana/App
cAdvisor & Node Exporter	Host Nodes	Node-level CPU, RAM usage, and container resource limits
Backup Exporter (9199)	Backup Service	Last execution age, duration, archive size, restore status

Table 6: Observability exporters and monitored metrics.

Figure 4 presents the main Grafana overview dashboard after simulating application traffic generated by user interactions with the reviews app.

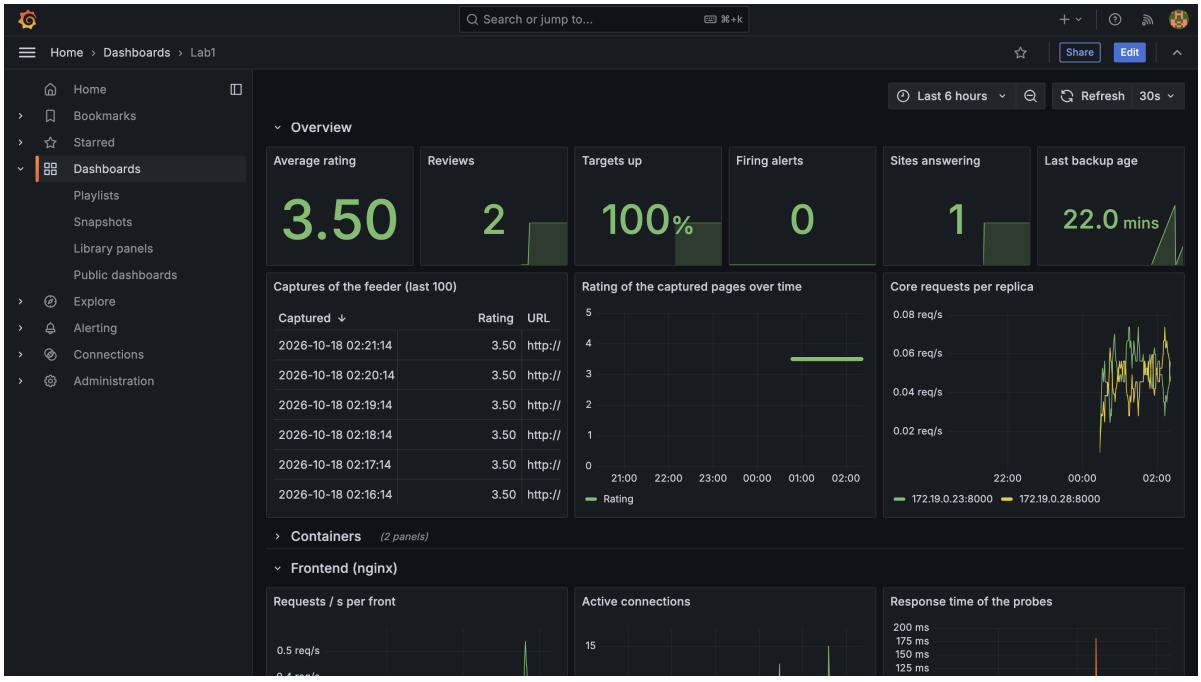


Figure 4: Grafana monitoring dashboard.

9 Security

- **Traffic Control**: Only Nginx exposes external ports (80/443). All traffic is forced through TLS 1.2/1.3 with HSTS enabled. Internal microservices remain isolated.

- **Least-Privilege Database Access**: Database owners (`app`, `grafana`) are restricted to their respective databases. The `replicator` user is confined to replication streams, and global `CONNECT` rights are revoked from `PUBLIC`.
- **Encrypted Inter-Node Communication**: Secrets are mounted in-memory at `/run/secrets`. Overlay networks utilize IPsec encryption, segregating individual application stacks.
- **Metrics Isolation**: Metric endpoints (`/metrics`) are blocked at the proxy layer, preventing external exposure.

10 Test cases

Each case says what to do, where to look and what must be seen. Commands are run from the project folder (local environment) or on the manager node, `front1` (production, `docker exec front1 ...`).

10.1 Local environment

Table 7: Test cases of the local environment.

	What we do	Where we look	Expected
L1	Publish a review in `app.localhost:8080`	`docker compose logs core | grep "POST /comment"`	status `302`; the review appears at the top and the counter grows
L2	(same review)	`docker compose logs smtp | grep status=sent`; page `mail.localhost:8080`	`status=sent`, and the mail "New review (n/5)" is listed
L3	Create a course in Atenea as `admin` (Site administration → Courses)	`docker compose logs moodle | grep "POST /course/edit.php"`; `docker compose exec moodle-db mariadb -umoodle -pgroupH moodle -e "select shortname from mdl_course"`	`303`; the course is in the table
L4	Stop the Postgres leader: `docker compose kill pg-1` (the leader)	`docker compose exec pg-2 patronictl -c /etc/patroni.yml list`; `docker compose logs pg-2 | grep promoted`	another node is *Leader*; "promoted self to leader"; a new review is still accepted
L5	Stop the Redis master: `docker compose stop redis-1`	`docker compose logs sentinel | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379`; the page keeps working
L6	Backup and restore test	`docker compose exec backup /backup.sh`; `docker compose exec backup /restore-test.sh`	`backup ok: /backups/...`; `restore test ok: n comments, 494 moodle tables`
L7	More replicas: `docker compose up -d -scale core=3`	several requests; header `X-App-Node`	the node name changes between requests
L8	Open Grafana, dashboard *Lab1*	panels of PostgreSQL, Redis, nginx, backups	data in every panel; one Patroni node with value 1

10.2 Production environment

Table 8: Test cases of the production environment.

	What we do	Where we look	Expected
P1	Deploy the stack	`docker node ls`; `docker service ls`; `docker stack ps lab1`	7 nodes *Ready*; every service at its replicas; each task on its node
P2	Open `https://app.localhost`	browser; `curl -k -D- https://app.localhost/`	the page loads after accepting the certificate; `X-Front-Node` and `X-App-Node` alternate between requests

	What we do	Where we look	Expected
P3	Publish a review	`docker service logs lab1_core | grep "POST /comment"`	`POST /comment ... 302` on one of the two core replicas
P4	(same review)	`http://mail.localhost:8080`; `docker service logs lab1_smtp | grep status=sent`	the mail is listed; `relay=mailpit[...]:1025 ... status=sent`
P5	Create a course in Atenea	`https://lms.localhost`; `docker service logs lab1_moodle | grep course/edit.php`; table `mdl_course` in the `lab1_moodle-db` container	`POST /course/edit.php ... 303`; new row (for example `PRUEBA1`)
P6	Lose a whole node: `docker stop app2`	`docker node ls`; `patronictl list`; `curl -k https://app.localhost/`	`app2` *Down* in ≈13 s; a replica becomes leader; the page answers 200 and a new review is accepted after the election (at most the 30 s `ttl`)
P7	Kill the Postgres leader container: `docker kill <pg-N container>`	`docker service logs lab1_pg-N | grep promoted`; `patronictl list`	"promoted self to leader because I had the session lock"; timeline (TL) grows by one; swarm restarts the container as a replica; no review is lost
P8	Kill the Redis master container	`docker service logs lab1_sentinel-1 | grep switch-master`	`+switch-master mymaster redis-1 6379 redis-2 6379` in about 2 s; the page keeps working
P9	Rolling update: `docker service update -force lab1_frontend`	a request every half second to `https://app.localhost/`	replicas replaced one by one; no request fails
P10	Backup on the backup node	`docker exec <lab1_backup container> /backup.sh` and `/restore-test.sh`	`backup ok ... source: db:5433`; `restore test ok`
P11	Restore the application database from a backup	write a review after the backup, then `scripts/restore.sh <folder> app` on the backup node; count the reviews	the review written after the backup disappears and the earlier ones are back; the tables keep their owner and the application keeps working
P12	Open Grafana	`https://grafana.localhost`, dashboard *Lab1*; Prometheus on the stats and backup nodes	36 targets *UP* in each Prometheus; panels with data

11 Results

11.1 Local environment

The whole stack starts with two commands (`docker compose pull` and `docker compose up -d`) and in less than a minute the 29 containers are healthy (Moodle needs about three minutes the first time, because it installs itself). The application works end to end: a review is stored in PostgreSQL, appears at once on the page, generates a mail that can be read in Mailpit, and updates the dashboards of Grafana. The database and the cache survive the loss of their main node: stopping the PostgreSQL leader makes another node take its place in a few seconds (about 3 s if the container is stopped, about 28 s if it is killed), and stopping the Redis master promotes its replica; in both cases the application keeps accepting reviews and no review is lost. The backup is encrypted and the automatic restore test recovers the reviews and the 494 tables of Moodle.

11.2 Production environment (`production.yml`)

Production is the same stack spread over seven nodes. It runs on our own computer: the nodes are seven Docker-in-Docker containers (`scripts/sim-swarm.sh`), three of them swarm managers. All the services reach their replicas and each one is on the node that its label asks for. What we obtained:

- **Entry:** the three sites answer on 443 through both front nodes (self-signed certificate) and port 80 redirects; requests are spread over front1/front2 and app1/app2.
- **Loss of a whole node (app2):** swarm detects it in 13 s, a database replica is promoted, the page keeps answering and reviews are accepted again once the election ends; when the node returns, its database rejoins as a replica without intervention.
- **Loss of a front node:** the other front serves all the sites.
- **Rolling update:** the frontend was replaced replica by replica while a request was sent every half second, and none failed.
- **Mail, Atenea and Grafana:** the mail of a new review is listed in `http://mail.localhost:8080`, a course created in Atenea is stored (log and database) and both Prometheus replicas see all their targets up.
- **Backups:** made from a database replica on the backup node, encrypted, and the restore test passes (reviews and 494 Moodle tables).

12 Limitations

- **Production is simulated on one computer.** The seven nodes are Docker-in-Docker containers, so the behaviour with the latency of separate machines is not measured. It is not opened to the Internet and has no public domain: the certificate is self-signed (the Let's Encrypt logic is in the stack but stays idle).
- **Quorum.** PostgreSQL and etcd need two of their three nodes; if two fail at the same time, writes stop until one returns. The third PostgreSQL node shares its machine with the monitoring.
- **Single instances.** MariaDB, the NFS server and the mail catcher have one instance (the statement exempts DBMS); losing the atenea node stops Moodle until it returns.
- **Backups** are kept on one disk of one node; there is no off-site copy.
- **Security simplifications of the laboratory.** All the secrets are `groupH` and are committed; NFS uses `no_root_squash` (limited to private networks) and its port must be closed in a firewall; the mail catcher has no login.

13 Conclusions

The goal of the project was to turn the infrastructure of a company into a set of small services that can be started anywhere with a couple of commands, and to prove that it keeps working when something breaks. We have a complete system: a reviews application, a learning platform, monitoring, mail and backups, all packaged as images that anyone can download and run, on one machine for development or on seven nodes for production.

The part that taught us most was not starting the services but making them survive failures. In a distributed system the pieces have to find each other, agree on who is in charge and recover without anybody touching them, and many of the problems we found appeared only when we switched machines off on purpose. Testing those failures from the beginning, and not at the end, saved us from believing that something worked just because it had started.

The result is a system where losing a web server, an application server, the cache or the database leader goes unnoticed by the user, and where the data is protected by encrypted copies that are checked automatically by restoring them. It is not a public service and it keeps some simplifications that a real company would not accept, which are listed in the previous section, but the structure is the same one that would be used with real machines: the next steps would be to move it to servers with a real domain, to keep a copy of the backups outside, and to manage the passwords properly.

Work distribution

Member 1	basic stack: frontend, cache, Let's Encrypt, NFS
Member 2	core app and interface, static files, Redis cluster
Member 3	PostgreSQL cluster (Patroni) and roles, SMTP relay, feeder, backups
Member 4	Atenea (Moodle, MariaDB)
Member 5	stats stack, exporters, swarm cluster and documentation